LABORATORY 4

DevTools & Debugging

inspector, breakpoints and the render tree

Mobile and Embedded Computing · Dinu-Ștefan Rusu

LABORATORY 4

What you'll build today

Attach the tools	Launch DevTools from the terminal and from your IDE, without looking it up
Find a real bug	Reproduce it, stop on it with a breakpoint, and read the state that caused it
Read the trees	Connect a pixel on screen to the widget, element and render object behind it
Explain a rebuild	Say which widgets rebuilt after a keystroke, and why the rest did not

This lab puts Lecture 4 to work. Debugging, setState, and the three trees: today you use them on code you wrote yourself.

STARTING POINT

Start from your Lab 3 todo app

Continue from the todo app you built in Laboratory 3. Do not start a new project.

Open that project, create a branch named lab4, and work there. Everything today happens in the file that holds your todo page, which for most of you is lib/main.dart or lib/todo_page.dart.

If your Lab 3 app never reached a working state, spend the first twenty minutes getting the four behaviors on the right to work. You cannot debug an app that does not run.

Commit that working version before you touch anything else. You will deliberately break this app, and you want something to compare against.

The app must already

add a todo from a text field

mark one done and undone

delete one

show how many are left

Nothing new is being built today. The app only has to run; the lab is about the tools.

STARTING POINT

The shape of the app you are debugging

State lives in the State object. Every change to it goes through setState. If your app is not shaped like this, make it so before Task I.

```dart
// lib/todo_page.dart
class Todo {
  Todo(this.title, {this.done = false});
  final String title;
  bool done;
}

class TodoPage extends StatefulWidget {
  const TodoPage({super.key});
  @override
  State<TodoPage> createState() => _TodoState();
}
```

```dart
class _TodoState extends State<TodoPage> {
  final _items = <Todo>[];
  final _input = TextEditingController();
  void _add() {
    final t = _input.text.trim();
    if (t.isEmpty) return;
    setState(() => _items.add(Todo(t)));
    _input.clear();
  }
  void _toggle(int i) => setState(
      () => _items[i].done = !_items[i].done);
  // dispose() the controller.
}
```

SETUP

Launching and attaching DevTools

```
# 1. Run the app in debug mode from the project folder.
flutter run

#     Flutter DevTools debugger and profiler is available at:
#     http://127.0.0.1:9100?uri=http://127.0.0.1:53412/AbCdEfGh=/
#     Cmd/Ctrl-click that URL, or paste the whole thing into a browser.

# 2. App already running and you closed the tab? Start DevTools
#     yourself, then paste the VM Service URI it asks for.
dart devtools

# 3. From the IDE. This is the only route that gives you breakpoints.
#     VS Code          F5, then Cmd/Ctrl+Shift+P -> "Dart: Open DevTools"
#     Android Studio   Run > Debug, then the Flutter Inspector tool window
```

Plain flutter run has no debugger attached. Start from the IDE and your breakpoints will be hit.

SETUP

The DevTools tabs you need today

Inspector

The live widget tree, Select Widget Mode, Track widget rebuilds and the Layout Explorer. Task III lives here.

Debugger

Breakpoints, the call stack and the Variables pane. Task II lives here, or in your IDE, which is faster.

Logging

Everything debugPrint writes, plus framework events. Keep it open all session.

Performance

The frame timeline and the performance overlay. Only meaningful in a --profile build.

Memory

Heap snapshots. This is how you find the controller you forgot to dispose.

Network

Every HTTP request the app makes. You will need this one in Lab 5.

TASK I

An interactive todo list

In your Lab 3 project, on branch lab4. Most of this exists already, so verify it and fill the gaps.

1. Confirm the todo page is a StatefulWidget and that the list of todos lives in its State object, not in the widget
2. Add a text field and an Add button that appends a new todo to the list inside setState
3. Make tapping a todo toggle its done flag inside setState
4. Add a delete action that removes a todo by index inside setState
5. Show the number of todos not yet done in the AppBar title
6. Commit this working version: it is your reference point for the rest of the lab

Hints

StatefulWidget and State<T>

setState(() { ... })

TextEditingController, and dispose() it

ListView.builder with itemCount

IconButton for the delete action

Done when

Add, toggle and delete all update the screen

No exceptions in the console

Committed on branch lab4

TASK II

The three bugs you will plant

Plant these one at a time. Fix each one before planting the next. Three simultaneous bugs teach you nothing.

```
// Bug 1: out-of-bounds index. Off by one, on purpose.
for (var i = 0; i <= _items.length; i++) { total += _items[i].done ? 1 : 0; }

// Bug 2: the model changes, the screen does not. No setState.
void _toggle(int i) => _items[i].done = !_items[i].done;

// Bug 3: a null that arrives later than you assumed.
String? _filter;                       // never assigned before the first build
final visible = _items.where((t) => t.title.contains(_filter!)).toList();
```

Bug 1 throws, bug 2 is silent, bug 3 throws somewhere else entirely. Three different failure shapes, three different tools.

TASK II

Break it, then find it with the debugger

Start the app from your IDE so the debugger is attached. Screenshot each bug while paused.

1. Plant bug 1 and run the app until it throws RangeError
2. Read the stack trace and set a breakpoint on the line it names
3. Step through the loop and watch i and _items.length in the Variables pane at the moment it fails
4. Fix the bound, and confirm the exception is gone
5. Plant bug 2, then use a conditional breakpoint to show the model changed while the screen did not
6. Add debugPrint calls logging the done flag before and after the mutation
7. Plant bug 3, reproduce the null failure, and fix it without using the ! operator

Hints

Click the gutter to set a breakpoint

Right-click it -> Edit condition

Condition: i == _items.length

debugPrint, not print

debugger() from dart:developer

Done when

All three bugs reproduced, then fixed

One screenshot per bug, paused at the breakpoint

Screenshots committed in /screenshots

REFERENCE

Breakpoints, conditions and debugPrint

A breakpoint pauses the isolate. While paused you can hover any variable, edit it in the Variables pane, and evaluate arbitrary Dart in the debug console.

A conditional breakpoint only stops when its expression is true: i == _items.length, or todo.title == 'milk'. Far more precise than a print inside a loop.

debugPrint throttles its output so Android does not silently drop lines; plain print will lose them when a loop is noisy.

Hot reload keeps your State objects; hot restart throws them away. If a fix does not appear on screen, that is nearly always the reason.

Step over

Run this line. Do not descend into the calls on it.

Step into

Descend into the call on this line.

Step out

Finish this function, stop back at the caller.

Resume

Run to the next breakpoint, or until the app exits.

A breakpoint you can leave in place beats a print you have to delete.

TASK III

Inspector, render tree and rebuild counts

With the fixed app running, open the Inspector tab in DevTools.

1. Turn on Select Widget Mode and tap a single todo row on the device
2. Screenshot the widget tree with that row selected, showing the widget the framework picked
3. Open the Layout Explorer on the Row or Column of that row and note the flex factors and incoming constraints
4. Call debugDumpRenderTree() from a button handler and capture the render tree it prints to the console
5. Turn on Track widget rebuilds, then type one single character into the text field
6. Write down which widgets rebuilt, and add one sentence to your README explaining why the rest did not

Hints

Inspector > Select Widget Mode

Track widget rebuilds (Inspector toolbar)

debugDumpRenderTree(), debugDumpApp()

import 'package:flutter/rendering.dart'

Layout Explorer for the yellow stripes

Done when

Screenshots of the widget tree and rebuild counts

Render tree output captured

Explanation committed in README.md

WHY IT WORKS

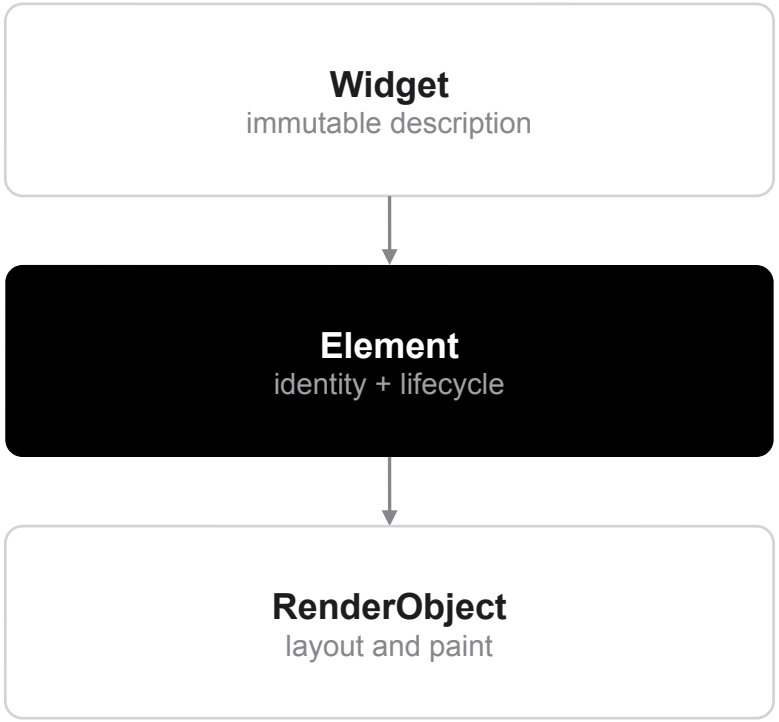
What actually rebuilds, and why

Flutter does not rebuild the whole widget tree each frame. setState marks one element dirty, and only that subtree is rebuilt on the next frame. Your rebuild counts should show exactly that.

Walking a rebuilt subtree, the framework reuses the existing element, and with it the State object and the render object, when the new widget has the same runtimeType and the same key. A different type, or a different key, and the old element is discarded.

That is why your list keeps its scroll position when one item changes, and why keys start to matter the moment items can be reordered or removed from the middle.

One frame budget is 1 ÷ refresh rate: 16.7 ms at 60 Hz, 8.3 ms at 120 Hz on most 2026 phones. There is no fixed 16 ms rule to memorize.



Rebuilt is not the same as repainted. The inspector counts rebuilds; the performance overlay shows what it cost.

TROUBLESHOOTING

When it goes wrong

	What it means	What to do
Breakpoints are hollow and never hit	The app is running without a debugger attached	Start it from the IDE (F5), not from a bare flutter run
DevTools opens but shows no data	It is pointed at a stale VM Service URI	Restart flutter run and open the URL it prints this time
The data changed, the screen did not	You mutated state outside setState	Track widget rebuilds will show zero rebuilds, which is your proof
RangeError: index out of range	You indexed at length, or the list shrank underneath you	Breakpoint on the indexing line; watch the index and the length
Yellow and black stripes on screen	A RenderFlex overflowed its constraints	Open the Layout Explorer on that Row or Column

Read the first line of the exception first. It names the type, the value and the file.

SUBMISSION

How to hand this in

Everything goes to your GitHub repository. No Word documents, no email, no Teams upload.

Commit your work to a branch named lab4, open a pull request, and merge it once it works.

Screenshots asked for in a task go in a /screenshots folder in the same repository.

Your commit history is part of the assessment. Commit as you go, not once at the end.

Checklist

- code committed and pushed
- screenshots where asked
- app builds and runs
- pull request merged

This lab is screenshot-heavy on purpose: a paused debugger and a live inspector are the only evidence that you used the tools rather than read the code.

Expected in /screenshots: one per planted bug paused at the breakpoint, one widget tree with a row selected, one Track widget rebuilds after a keystroke.

Wrapping up

Recap

DevTools attaches from the terminal or the IDE, but only the IDE route gives you breakpoints

setState marks one element dirty; the subtree rebuilds, not the app

Element reuse is decided by runtimeType, then key

Before next lab

Push branch lab4 and open the pull request

Screenshots in /screenshots, nowhere else

Skim Lecture 5: HTTP, JSON and code generation

Read more

docs.flutter.dev/tools/devtools

docs.flutter.dev/tools/devtools/inspector

api.flutter.dev: debugDumpRenderTree, debugPrint